

Data Engineering & ETL

Interview Preparation Guide

100 Generic Questions — No Tool-Specific Assumptions

Covers: Data Pipelines • ETL/ELT • Batch & Streaming • Data Quality • Storage • Governance • Architecture

FAANG / High-Scale Startup Level

SECTION 1

SECTION 1 – Direct Conceptual Questions (Q1 – Q45)

45 questions covering core data engineering fundamentals, ETL concepts, storage systems, pipeline design, and data processing — all tool-agnostic.

Q1. What is the difference between ETL and ELT, and when would you choose one over the other?

Answer:

ETL (Extract → Transform → Load) transforms data before loading it into the destination. **ELT (Extract → Load → Transform)** loads raw data first, then transforms inside the destination system.

Explanation:

ETL was the standard when destination systems (warehouses) were expensive and could not handle heavy compute. Today, cloud warehouses have massive compute power, making ELT practical — raw data lands first, and transformation happens inside the warehouse using SQL. The main advantage of ELT is flexibility: you can change transformation logic at any time without re-extracting the source data.

When to choose:

- ETL: Source data is sensitive and must be masked before reaching the destination. Destination system has limited compute capacity.
- ELT: Modern cloud warehouse with strong compute. Need to preserve raw data for re-processing. Transformation logic changes frequently.

Q2. What is an idempotent data pipeline and why is it critical in production?

Answer:

Idempotency means running a pipeline once or multiple times produces the exact same result in the destination. No extra rows, no duplicates, no partial data — the outcome is always consistent.

Explanation:

In production, pipelines fail and are retried. Without idempotency, a retry could insert duplicate records or load partial data on top of already-loaded data. Idempotent pipelines are safe to retry because they clean up and reload correctly every time.

How to achieve idempotency:

- Delete the target partition before reloading it (delete-then-insert pattern).
- Use UPSERT / MERGE operations keyed on a unique identifier.
- Write to a staging table first, then atomically swap to the final table.

Example:

Pipeline loads daily sales. Before inserting, it deletes all rows for that date. Even if it runs twice, the result is always one correct copy of that day's data.

Q3. What is incremental data loading and how does it differ from a full load?

Answer:

Type	What it does	When to use	Trade-off
Full Load	Truncate the destination and reload all rows from scratch	Small tables, initial historical load	Expensive for large tables
Incremental Load	Only load new or changed records since the last successful run	Large tables updated frequently	Requires change tracking in source

Explanation:

Incremental loading uses a watermark — typically a timestamp column like `updated_at` or a sequential integer ID. The pipeline records the last successfully processed watermark value and on the next run, only fetches records newer than that value. This reduces time, cost, and load on the source system.

Example:

Query: `SELECT * FROM orders WHERE updated_at > '2024-06-01 08:00:00'`. Only new or recently changed orders are fetched, not all 10 million historical rows.

Q4. What is a data pipeline and what are its main stages?

Answer:

A **data pipeline** is an automated sequence of steps that moves data from source systems to a destination, applying transformations and validations along the way.

Main stages:

- Extract: Pull data from source systems — databases, APIs, files, event streams.
- Validate / Stage: Check schema, data types, and basic quality. Store raw data in a staging area.
- Transform: Apply business logic — clean, enrich, aggregate, join, deduplicate.
- Load: Write processed data to the destination — data warehouse, data lake, or another database.
- Monitor & Alert: Track pipeline health, data quality, and SLA compliance.

Analogy:

Think of a pipeline like a factory assembly line. Raw materials come in (extract), are shaped and refined (transform), inspected for quality (validate), and the finished product is stored in a warehouse (load).

Q5. What is the difference between batch processing and stream processing?**Answer:**

Aspect	Batch Processing	Stream Processing
When data is processed	After data accumulates (hourly/daily)	As data arrives (real-time / near-real-time)
Latency	High — minutes to hours	Low — milliseconds to seconds
Complexity	Simpler to implement and debug	More complex — late data, ordering, state
Use case	Daily reports, historical ETL jobs	Fraud detection, live dashboards, alerts
Error handling	Retry the whole batch	Must handle per-event errors carefully

Analogy:

Batch is like doing laundry once a week — you accumulate items and process them together. Streaming is like washing each item immediately after use. Choose based on how fresh your data needs to be.

Q6. What is a data lake and how is it different from a data warehouse?**Answer:**

Feature	Data Lake	Data Warehouse
Storage format	Raw — any format (JSON, CSV, Parquet, images)	Structured, processed, modeled data
Schema approach	Schema-on-read (defined when queried)	Schema-on-write (defined when loaded)
Primary users	Data engineers, data scientists	Business analysts, BI tools
Cost	Cheap — cloud object storage (S3, GCS)	More expensive — compute + storage
Data quality	Raw, may be messy	Curated, trusted, business-ready

Explanation:

A data lake stores everything cheaply in its raw form — you decide how to structure it later. A data warehouse stores clean, modeled, analytics-ready data. Modern platforms use both: the data lake as a raw storage and processing layer, feeding curated data into the warehouse for business analytics.

Q7. What is partitioning in data engineering and why is it important?**Answer:**

Partitioning divides a large dataset or table into smaller, independent chunks (partitions) based on the value of a column — most commonly a date column.

Why it matters:

- Query performance: A query filtering on the partition column only scans matching partitions, skipping all others.
- Pipeline efficiency: Incremental jobs process only the new partition, not the full table.
- Cost reduction: In pay-per-scan systems, scanning 1 partition instead of 365 is 365x cheaper.
- Easier maintenance: Delete, reload, or repair one partition without touching the rest.

Example:

A sales table partitioned by `sale_date`. A query for last week only reads 7 partitions out of 3 years of data. Without partitioning, it scans every single row.

Q8. What is schema evolution and how do you handle it in a data pipeline?

Answer:

Schema evolution refers to changes in the structure of source data over time — new columns added, columns removed, data types changed, or columns renamed.

Common scenarios:

- A new column appears in the source API response.
- A column is renamed from `'customer_id'` to `'user_id'`.
- A field changes from string to integer.

Handling strategies:

- Additive changes (new columns): Most systems handle this gracefully — add the new column as nullable in the destination.
 - Breaking changes (rename, type change): Require a migration plan — create a new version of the table or apply a transformation to standardize.
 - Schema validation: Check incoming data against an expected schema at the start of the pipeline. Alert on unexpected changes before they cause downstream failures.
 - Preserve raw data: Always store raw source data before applying schema transformations so you can re-process if your schema handling logic is wrong.
-

Q9. What is a staging area in an ETL pipeline and why is it important?

Answer:

A **staging area** is a temporary intermediate storage zone where raw extracted data is held before transformation and final loading into the destination.

Why it is important:

- Recovery: If the transformation step fails, you re-run only the transform — no need to re-extract from the source (which may be slow or rate-limited).
- Auditing: Raw data is preserved for debugging and compliance — you can always trace what the source sent.
- Decoupling: Extract and transform steps are independent. Each can be optimized, retried, or scaled separately.
- Data quality: You can run validation checks on staged data before it enters the final destination.

Typical flow:

Extract → Raw staging (cloud storage or staging schema) → Validate → Transform → Load to final table.

Q10. What is columnar storage and why is it preferred for analytical workloads?

Answer:

Columnar storage organizes data by column rather than by row. All values of a column are stored together on disk, instead of all columns of a row being stored together.

Why it is better for analytics:

- **Selective reading:** A query `SELECT SUM(revenue)` only reads the revenue column. Row storage reads every column of every row even when only one column is needed.
- **Better compression:** Values in the same column are often similar (e.g., a country column has many repeated values). Similar values compress far better than mixed row data.
- **Faster aggregations:** Summing or averaging a column is a sequential read of one contiguous block — extremely cache-friendly.

Example:

A table with 100 columns and 1 billion rows. Query: `SELECT AVG(age)`. Row storage reads all 100 billion column values. Columnar reads only 1 billion age values — 100x less data.

Q11. What is the difference between OLTP and OLAP systems?

Answer:

Feature	OLTP	OLAP
Purpose	Handle live application transactions	Handle analytical queries and reports
Query pattern	Many short queries on few rows	Few long queries scanning many rows
Data model	Normalized (reduce redundancy)	Denormalized (reduce joins)
Latency	Milliseconds	Seconds to minutes
Examples	PostgreSQL, MySQL, Oracle	Redshift, BigQuery, Snowflake

Why it matters to data engineers:

You never run heavy analytics directly on an OLTP database — it would slow down the production application. ETL pipelines exist precisely to move data from OLTP (source of truth) into OLAP (analytics system) on a regular schedule.

Q12. What is a slowly changing dimension (SCD) and what are the most common types?

Answer:

Slowly Changing Dimensions (SCDs) are dimension tables (like customer, product, location) where attribute values change over time but slowly and infrequently.

Type	Strategy	History kept?	Use case
SCD Type 1	Overwrite old value with new value	No	When history is not important (e.g., fixing a typo)
SCD Type 2	Insert a new row with validity dates	Full history	When historical accuracy matters (e.g., customer address at time of purchase)
SCD Type 3	Add a 'previous_value' column	One level only	When only one previous value is needed

Example:

Customer moves from Mumbai to Delhi. SCD1: Overwrite city = Delhi (Mumbai history is lost). SCD2: Keep Mumbai row as expired, add new Delhi row with valid_from date. Historical orders now correctly link to the Mumbai address.

Q13. What is a surrogate key and why is it preferred over natural keys in a data warehouse?**Answer:**

A **surrogate key** is a system-generated, meaningless integer (1, 2, 3...) used as the primary key in a dimension table, replacing the natural business key (email, SKU, phone number).

Why surrogate keys are better:

- Natural keys can change: A customer changes their email. A product SKU gets renamed. Surrogate keys never change.
- Enables SCD Type 2: Multiple rows for the same entity can have different surrogate keys, allowing full history tracking.
- Performance: Integer joins are faster than string joins.
- Independence: The warehouse is not coupled to source system key formats or conventions.

Example:

Customer email changes from alice@old.com to alice@new.com. With surrogate key (customer_sk = 1001), all historical fact records remain correctly joined. With natural key (email), all historical joins break.

Q14. What is a star schema and why is it used in data warehouses?**Answer:**

A **star schema** has one central fact table containing measurable business events (sales, clicks, payments) surrounded by dimension tables (customer, product, date, location) connected by foreign keys. The shape looks like a star.

Why it is used:

- Simple structure: Easy for business analysts and BI tools to understand without deep SQL knowledge.
- Query performance: Fewer joins than a fully normalized model. Fact table joins directly to each dimension.
- BI tool friendly: Tableau, Power BI, Looker work optimally with star schemas.

Example:

Fact table: fact_sales (order_id, customer_id, product_id, date_id, revenue, quantity). Dimensions: dim_customer, dim_product, dim_date. A query for 'revenue by product category by month' joins only fact_sales + dim_product + dim_date.

Q15. What is data lineage and why does it matter in a data platform?**Answer:**

Data lineage tracks the complete journey of data — where it originated, which pipelines processed it, what transformations were applied, and where it ended up. It is the full audit trail of data movement.

Why it matters:

- **Debugging:** When a dashboard shows wrong numbers, lineage lets you trace back through every transformation to find where the error was introduced.
- **Impact analysis:** Before changing a source table, understand which downstream pipelines, reports, and models will be affected.
- **Compliance:** Prove to auditors where sensitive data came from and how it was handled (GDPR, HIPAA).
- **Trust:** Analysts trust data more when they can see exactly where it came from and how it was processed.

Example:

CEO sees wrong revenue number → lineage shows: raw_orders → cleaned_orders → daily_revenue → revenue_dashboard. Engineer traces each step to find the transformation that introduced the error.

Q16. What is the difference between data normalization and denormalization, and when do you use each?**Answer:**

Normalization reduces data redundancy by organizing data into multiple related tables.

Denormalization combines tables to reduce the number of joins needed for queries.

Aspect	Normalization	Denormalization
Goal	Eliminate redundancy, ensure consistency	Improve query read performance
Tables	Many smaller, related tables	Fewer, wider tables
Joins	Many joins required for queries	Fewer joins required
Used in	OLTP (production databases)	OLAP (data warehouses)
Trade-off	Slower reads, faster writes	Faster reads, more storage

Key principle:

Normalize to prevent update anomalies in transactional systems. Denormalize in analytical systems where you read far more than you write, and query speed is the priority.

Q17. What is a data catalog and why is it important for a data team?**Answer:**

A data catalog is a centralized, searchable inventory of all data assets in an organization — listing what tables and columns exist, what they mean, who owns them, when they were last updated, and how they relate to each other.

Why it is critical:

- Discoverability: Engineers and analysts can find the right data without asking around or guessing table names.
- Understanding: Column descriptions explain what each field means in business terms, not just technical names.
- Governance: Tag sensitive columns (PII, financial) for access control and compliance tracking.
- Trust: Show data quality scores, freshness, and ownership so consumers know whether to trust a dataset.

Without a catalog:

Teams waste hours searching for data, accidentally use deprecated tables, build duplicate pipelines, and make decisions on data they do not fully understand.

Q18. What is a dead letter queue (DLQ) in a data pipeline and when do you use it?

Answer:

A dead letter queue (DLQ) is a separate storage location (queue, table, or file path) that captures records that a pipeline failed to process successfully after multiple retries — instead of losing them or blocking the pipeline.

Why it is useful:

- Continuity: The pipeline keeps processing healthy records instead of getting stuck on one bad record.
- No data loss: Bad records are not silently dropped — they are parked in the DLQ for inspection.
- Debugging: Engineers inspect DLQ records to identify data quality issues or bugs.
- Reprocessing: After fixing the bug, records in the DLQ can be replayed through the pipeline.

Example:

An event fails schema validation 3 times. Instead of crashing the pipeline or skipping silently, it is written to a 'failed_events' table with the error reason, timestamp, and original payload.

Q19. What is data quality and what are the dimensions used to measure it?

Answer:

Data quality measures how fit a dataset is for its intended use. Poor data quality leads to wrong analysis, bad decisions, and broken pipelines.

Key dimensions of data quality:

- Completeness: Are all required fields populated? Are there unexpected NULLs?
- Accuracy: Do values reflect reality? Is revenue matching the source system?
- Consistency: Does the same entity have the same value across different tables?
- Timeliness / Freshness: Is data available by the expected time? How recent is it?
- Uniqueness: Are there duplicate records for the same entity?
- Validity: Do values conform to expected formats and ranges? (e.g., age cannot be -5 or 999)

In practice:

Build automated checks for each dimension as part of your pipeline. Alert on failures before bad data reaches business users.

Q20. What is the difference between data validation and data testing in a pipeline?**Answer:**

Aspect	Data Validation	Data Testing
What it checks	The actual data values at runtime	The pipeline code and transformation logic
When it runs	Every time the pipeline runs (on live data)	During development and CI/CD (on test data)
Goal	Catch bad data before it reaches consumers	Catch bugs in transformation logic before deployment
Example	Check that revenue > 0 and customer_id is never NULL	Assert that a transformation produces the expected output for known inputs

Best practice:

You need both. Data testing catches bugs in code before deployment. Data validation catches bad source data in production even when the code is correct. One cannot replace the other.

Q21. What is a data contract and how does it improve reliability between pipeline teams?**Answer:**

A **data contract** is a formal agreement between a data producer (team that generates data) and data consumers (teams that use it), defining the schema, data types, quality expectations, and SLAs of a dataset.

What a data contract typically includes:

- Schema: Column names, data types, nullability.
- Semantics: Business definition of each field.
- SLA: When will data be available? What is the freshness guarantee?
- Quality: What guarantees are made (no nulls in key columns, unique primary keys)?
- Ownership: Who is responsible for maintaining this data?

Why it matters:

Without contracts, producers change schemas without warning and silently break downstream consumers. Contracts enforce communication and accountability, reducing data incidents.

Q22. What is the fan-out problem in data pipelines and how do you address it?**Answer:**

Fan-out occurs when a single upstream dataset or pipeline is consumed by many downstream pipelines. When the upstream breaks or changes, all downstream consumers break simultaneously.

Why it is a problem:

Imagine a central 'clean_events' table consumed by 20 downstream jobs. A schema change or data quality issue in that table cascades into 20 simultaneous failures across different teams.

Solutions:

- Data contracts: Formalize what the upstream guarantees so changes are communicated before they happen.
- Intermediate staging tables: Each consumer reads from a dedicated copy or view, not directly from the shared upstream.
- Versioned datasets: When a breaking change is necessary, publish a new version (v2 table) alongside the old one during a migration period.
- Impact analysis: Before making any upstream change, identify all consumers through lineage tracking.

Q23. What is the difference between a fact table and a dimension table?

Answer:

Aspect	Fact Table	Dimension Table
Contains	Measurable business events (metrics)	Descriptive context (attributes)
Examples of columns	revenue, quantity, duration, count	customer_name, product_category, date, city
Row represents	One business event (a sale, a click)	One entity (one customer, one product)
Size	Very large — grows constantly	Smaller — changes slowly
Key type	Foreign keys + measures	Primary key + descriptive attributes

Example:

fact_orders contains: order_id, customer_id (FK), product_id (FK), date_id (FK), revenue, quantity.
dim_customer contains: customer_id (PK), name, city, segment. Joining them answers: 'What is total revenue by customer city?'

Q24. What is change data capture (CDC) and how does it work conceptually?

Answer:

CDC (Change Data Capture) is a technique that detects and captures row-level changes (INSERT, UPDATE, DELETE) from a source database and propagates them to downstream systems without scanning the full table.

How it works conceptually:

- Log-based CDC: Reads the database's internal transaction log (which records every change). No extra load on the source database. This is the gold standard approach.
- Trigger-based CDC: Database triggers fire on each change and record it to a change log table. Adds overhead to every write on the source.
- Polling-based CDC: Periodically query for records where updated_at > last_run_time. Simple but cannot detect hard deletes.

Why CDC is valuable:

Instead of running expensive full-table scans every hour to find what changed, CDC captures changes in real time with minimal impact on the source system. It enables near-real-time data pipelines without polling.

Q25. What is a watermark in the context of data pipelines and why is it important?

Answer:

A **watermark** (also called a high-water mark) is a stored checkpoint value — usually a timestamp or an ID — that records the furthest point successfully processed by a pipeline.

How it works:

After each successful pipeline run, the watermark value is updated to the maximum timestamp or ID processed. On the next run, the pipeline fetches only records newer than the stored watermark. This is the foundation of incremental loading.

Why it is important:

- **Efficiency:** Process only new data, not the full history every time.
- **Recovery:** If a run fails midway, the watermark hasn't been updated, so the next run re-processes the same window — no data is skipped.
- **Auditability:** The watermark log shows exactly what has been processed and when.

Example:

Watermark stored: 2024-06-01 08:00:00. Next run fetches: `WHERE updated_at > '2024-06-01 08:00:00'`. After success, watermark is updated to 2024-06-01 09:00:00.

Q26. What are the main challenges of handling duplicate records in a data pipeline?

Answer:

Why duplicates occur:

- **At-least-once delivery:** A pipeline retried after a partial failure and inserted the same records again.
- **Source system sends duplicates:** An API or event system sends the same event more than once.
- **Multiple pipeline runs overlap:** Two runs process the same time window simultaneously.

Why duplicates are harmful:

Duplicates silently inflate every metric — revenue appears doubled, user counts are inflated, reports are wrong. They are often hard to detect because the numbers look 'reasonable' rather than obviously wrong.

Strategies to handle duplicates:

- **UPSERT / MERGE:** Insert-or-update based on a unique key. Duplicate records update the existing row instead of adding a new one.
- **Deduplication query:** Use `ROW_NUMBER() OVER (PARTITION BY id ORDER BY updated_at DESC) = 1` to keep only the latest version of each record.
- **Idempotent partition overwrite:** Delete the partition and reload — even if loaded twice, result is always one copy.
- **Unique constraint enforcement:** Add a unique constraint on the primary key in the destination table so duplicates are rejected at the database level.

Q27. What is data freshness and how do you monitor it in production?

Answer:

Data freshness measures how recent the data in a table or dashboard is — the lag between when an event happened in the source system and when it is visible in the data platform.

Why it matters:

Stale data leads to wrong decisions. If a sales dashboard is 12 hours behind, a manager making real-time decisions is working with yesterday's reality. Every table should have a freshness SLA (e.g., 'data available by 8 AM daily').

How to monitor freshness:

- Track MAX(loaded_at) or MAX(updated_at) on every critical table.
- Alert when MAX(loaded_at) is older than the SLA threshold (e.g., data is more than 2 hours old).
- Add freshness checks to your pipeline monitoring dashboard.
- Run automated freshness tests daily — fail the check if data has not been updated within the expected window.

Q28. What is the difference between structured, semi-structured, and unstructured data?**Answer:**

Type	Definition	Examples	Storage
Structured	Fixed schema, rows and columns	SQL tables, CSV	Relational DB, warehouse
Semi-structured	Flexible schema, self-describing	JSON, XML, Avro, Parquet	Data lake, NoSQL DB
Unstructured	No predefined format	Images, video, PDFs, emails	Object storage (S3, GCS)

Relevance to pipelines:

ETL pipelines often receive semi-structured data (JSON API responses) and transform it into structured tables for analytics. Unstructured data (documents, images) often requires machine learning models to extract meaningful signals before it can be analyzed as structured data.

Q29. What is the purpose of data reconciliation in a pipeline and how do you implement it?**Answer:**

Data reconciliation is the process of comparing data in the destination system against the source system to confirm that all records were transferred correctly, completely, and accurately.

Why it is essential:

Pipelines can silently drop records due to filtering bugs, type conversion failures, or network issues. Without reconciliation, you would never know. Reconciliation catches these gaps before business decisions are made on incomplete data.

How to implement:

- Row count check: Source row count for a given date must equal destination row count for the same date.
- Sum/aggregate check: SUM(revenue) in source must equal SUM(revenue) in destination.
- Null check: Critical columns (customer_id, order_id) must have zero NULLs in the destination.
- Sample spot check: Randomly select N records from source and verify they exist with correct values in the destination.

When to run:

After every pipeline run. Automate the reconciliation check as a pipeline task that runs after the load step. Fail the pipeline if reconciliation fails.

Q30. What is a pipeline dependency and what problems does it cause when not managed properly?

Answer:

A pipeline dependency exists when one pipeline or task must complete successfully before another can start. Unmanaged dependencies lead to data corruption, incorrect results, or cascading failures.

Problems of unmanaged dependencies:

- Race conditions: Pipeline B starts before Pipeline A finishes. B reads incomplete data from A and produces wrong results.
- Cascading failures: A fails but B still runs, processing stale data. B succeeds, loading wrong results into the warehouse.
- Hidden coupling: Two pipelines implicitly depend on each other but nobody documented it. A change in one silently breaks the other.

Best practices:

- Explicitly declare all dependencies in your orchestration tool.
 - Use sensors or triggers — Pipeline B only starts when Pipeline A signals success.
 - Document upstream dependencies for every pipeline and review impact before making changes.
-

Q31. What is late-arriving data and how do you design pipelines to handle it?

Answer:

Late-arriving data refers to records that arrive in a pipeline after the expected processing window has already closed — for example, an event from yesterday that is received and processed today.

Why it happens:

- Network delays, retries, or system outages in the source.
- Mobile devices that were offline and sync data hours or days later.
- Batch source systems that deliver data with a lag.

Strategies to handle late data:

- Reprocessing window: Instead of trusting only today's data, always reprocess the last N days (e.g., last 3 days) in each pipeline run to capture late arrivals.
 - Partition by event time: Store data in partitions based on when the event actually occurred, not when it was loaded. Late records land in the correct historical partition.
 - Append + deduplicate: Accept late arrivals freely. Run a daily deduplication job to produce a clean final view.
 - Flagging: Mark records as 'late' so downstream consumers know they are corrections to previously processed data.
-

Q32. What is data governance and what role does a data engineer play in it?

Answer:

Data governance is the set of policies, standards, processes, and ownership rules that ensure data is accurate, consistent, secure, compliant, and properly used across an organization.

Key components:

- Data ownership: Every dataset has a named owner responsible for its quality and maintenance.
- Access control: Who is allowed to see which data? Especially critical for PII and financial data.
- Data quality standards: What quality thresholds must each dataset meet?
- Compliance: Ensure pipelines handle data according to regulations (GDPR, HIPAA, CCPA).
- Metadata and lineage: Document data origins, transformations, and definitions.

Data engineer's role:

Implement the technical mechanisms — access controls, masking policies, lineage tracking, quality checks, retention policies — that support the governance policies defined by the organization.

Q33. What is the difference between hot, warm, and cold data in a storage strategy?**Answer:**

Tier	Definition	Access frequency	Storage cost	Example
Hot	Actively queried data	Daily / real-time	Highest	Last 30 days in warehouse
Warm	Occasionally queried data	Weekly / monthly	Medium	Last 1 year in data lake
Cold	Rarely accessed, archival data	Yearly or never	Lowest	Data older than 3 years in archive storage

Why this matters:

Storing all data in the most expensive hot tier is wasteful. A tiered storage strategy automatically moves data to cheaper tiers as it ages, dramatically reducing storage costs. Most data access follows the 80/20 rule — 80% of queries hit only the most recent 20% of data.

Q34. What is metadata management and what types of metadata does a data engineer typically work with?**Answer:**

Metadata is data about data — it describes the structure, origin, quality, and usage of datasets without containing the actual data values.

Types of metadata:

- Technical metadata: Column names, data types, row counts, table sizes, last updated timestamp, schema version.
- Business metadata: Column descriptions in plain English, business rules, what a metric means, which team owns the data.
- Operational metadata: Pipeline run history, job duration, SLA compliance, error counts, data freshness.
- Governance metadata: PII flags, data sensitivity classification, access control lists, retention policies.

Why good metadata management matters:

Without metadata, engineers and analysts waste enormous time figuring out what data exists, what it means, and whether it can be trusted. Good metadata is the foundation of a self-service data platform.

Q35. What is a materialized view and when should you use one in a data pipeline?

Answer:

A **materialized view** is a database object that stores the pre-computed result of a query physically on disk, unlike a regular view which re-executes the query every time it is accessed.

When to use a materialized view:

- An expensive aggregation query is run frequently (e.g., daily revenue summary queried hundreds of times per day).
- A complex multi-table join is used as a base for many downstream reports.
- Query latency must be low but perfect real-time freshness is not required.

Trade-offs:

Materialized views consume storage space and must be refreshed on a schedule or manually. They show stale results between refreshes. The key question is: 'Is the query expensive enough and frequent enough that pre-computing it is worth the storage and refresh cost?'

Q36. What is schema-on-read vs schema-on-write and when is each appropriate?

Answer:

Aspect	Schema-on-Write	Schema-on-Read
Definition	Schema is enforced when data is written	Schema is applied when data is read / queried
Data stored as	Structured, pre-modeled	Raw (JSON, Parquet, files)
Flexibility	Low — schema changes require migration	High — raw data supports multiple schemas
Query speed	Fast — data is already structured	Slower — parsing happens at query time
Used in	Relational DB, data warehouse	Data lake, Hadoop, NoSQL

Explanation:

Schema-on-write is predictable and fast for known query patterns. Schema-on-read is flexible — you can apply different schemas to the same raw data for different use cases. Modern data platforms often use schema-on-read in the raw lake zone and schema-on-write in the curated warehouse zone.

Q37. What is data skew in distributed data processing and how does it affect pipeline performance?

Answer:

Data skew occurs when data is unevenly distributed across processing partitions. One partition gets significantly more data than others, creating a bottleneck that slows the entire job.

Why it happens:

- A join or group-by key has a highly unequal value distribution (e.g., 80% of rows have country = 'US').
- NULL values in a join key all land on the same partition.

- Poor partitioning choice — partitioning by a low-cardinality column.

Impact:

In distributed processing, all workers must finish before the job moves to the next step. One overloaded partition makes every other worker wait idle. A job that should take 10 minutes takes 2 hours because of one skewed partition.

Generic solutions:

- Salting: Add a random prefix to the skewed key to artificially distribute it across more partitions.
- Pre-aggregate: Reduce data volume before the join/group-by to reduce the skew impact.
- Better partitioning: Choose a higher-cardinality partition key.

Q38. What is connection pooling and why is it important in data pipelines?**Answer:**

Connection pooling maintains a set of pre-established, reusable database connections. Instead of opening and closing a new connection for every query, pipeline tasks borrow connections from the pool and return them when done.

Why creating a new connection is expensive:

Each new database connection requires a TCP handshake, authentication, session initialization, and memory allocation on the server. In a pipeline with 500 tasks running in parallel, creating 500 simultaneous new connections can crash the database.

Benefits of connection pooling:

- Performance: Connection reuse eliminates repeated handshake overhead.
- Stability: Limits the maximum number of simultaneous database connections from the pipeline.
- Resource efficiency: Fewer connections = less memory and CPU on the database server.

Practical tip:

Configure your pipeline's connection pool size to match what the database can handle — typically 10–50 connections per service. Too few = bottleneck. Too many = database overloaded.

Q39. What is the difference between hard deletes and soft deletes and how does each affect data pipelines?**Answer:**

Type	How it works	Pipeline challenge
Hard delete	Row is physically removed from the source database	Incremental pipelines based on updated_at miss hard deletes — deleted rows never show up in the next run
Soft delete	A deleted_at or is_deleted column is set instead of removing the row	Row appears in next incremental load with the deleted flag — pipeline can process the deletion correctly

Why this matters:

If your source system uses hard deletes and your pipeline only does incremental loads based on updated_at, your destination will forever contain records that were deleted from the source. Solutions:

use CDC (log-based) to capture deletes, run periodic full reconciliation to detect ghost records, or request the source team to implement soft deletes.

Q40. What is a snowflake schema and how does it differ from a star schema?

Answer:

Aspect	Star Schema	Snowflake Schema
Dimension structure	Flat — one denormalized dimension table	Normalized — dimension split into sub-tables
Number of tables	Fewer tables	More tables
Joins required	Few — fact joins directly to each dimension	More — must join through sub-dimension tables
Query performance	Faster queries	Slightly slower due to more joins
Storage	More storage (redundancy)	Less storage (no redundancy)

Explanation:

In a star schema, `dim_product` has `product_id`, `name`, `category_name`, `subcategory_name` all in one table. In a snowflake schema, `category_name` lives in a separate `dim_category` table that `dim_product` references. Snowflake is more normalized. Star schema is more query-friendly. Most modern data warehouses prefer star schema for analytical workloads.

Q41. What is the difference between a push-based and pull-based data ingestion pattern?

Answer:

Pattern	How it works	Who initiates	Use case
Pull-based	The pipeline goes to the source and fetches data on a schedule	Consumer (pipeline)	Batch ETL from databases, APIs, files
Push-based	The source system sends data to the pipeline whenever something happens	Producer (source)	Real-time webhooks, event streams, CDC

Trade-offs:

- Pull is simpler to implement and control — you choose when and how much to fetch. But you may miss data if the poll interval is too long, and you add load to the source with every poll.
- Push is more real-time and reduces source polling load. But requires the source to support push mechanisms (webhooks, event publishing) and the consumer to be always ready to receive.

Most production systems:

Use pull for batch ETL from databases and APIs. Use push (CDC, webhooks) for near-real-time use cases.

Q42. What is data masking and when is it used in data pipelines?

Answer:

Data masking replaces sensitive data values with realistic but fictitious values — protecting PII and confidential information while preserving the data structure and format for development, testing, or analytics.

Types of masking:

- **Static masking:** Permanently replace sensitive values in a dataset (e.g., production data copied to staging has all emails replaced with fake ones).
- **Dynamic masking:** Real values stored in the database but masked at query time based on user role. Analysts see 'XXX-XX-1234' while the actual SSN is stored.
- **Tokenization:** Replace sensitive value with a consistent token. Same email always maps to the same token — allows joins and aggregations without exposing the real value.

When to use:

Whenever copying production data to lower environments (dev, staging, analytics). Whenever exposing data to third parties or external analysts. Whenever GDPR, HIPAA, or PCI-DSS compliance is required.

Q43. What is the difference between event time and processing time in data pipelines?

Answer:

Concept	Definition	Example
Event time	When the event actually occurred in the real world	User clicked 'Buy' at 10:03 AM
Processing time	When the pipeline received and processed the event	Pipeline processed it at 10:17 AM (14 min delay)

Why the difference matters:

If your hourly report uses processing time, a spike in system latency at 10:05 AM means events from 10:00–10:10 are counted in the 10:10–11:00 bucket instead of the correct 10:00–11:00 bucket. This produces incorrect time-based analytics. Always store event time in your data and aggregate by event time for accurate historical analysis.

Practical rule:

Always capture both `event_time` (from the source) and `loaded_at` (when the pipeline ran). Use `event_time` for analytics. Use `loaded_at` for pipeline monitoring and debugging.

Q44. What are data retention policies and how do they affect pipeline design?

Answer:

Data retention policies define how long different types of data must be kept and when it should be deleted or archived. They are driven by business needs, cost management, and legal/compliance requirements.

How retention policies affect pipelines:

- **Automated deletion:** Pipelines must include a cleanup step that deletes or archives data older than the retention threshold.
- **GDPR / right to erasure:** If a user requests deletion, the pipeline must identify and delete all records containing that user's PII across all tables — including historical and archived data.

- Storage tiering: Data approaching the retention limit can be moved to cheaper cold storage before deletion.
- Backfill limits: Retention policies cap how far back a pipeline can backfill. Data older than the retention period no longer exists.

Example:

Raw event logs retained 90 days. Daily aggregated reports retained 7 years. User PII retained until account deletion + 30 days. Each type of data may have different retention rules requiring different pipeline behaviors.

Q45. What is the role of a data engineer vs a data scientist vs a data analyst?**Answer:**

Role	Primary focus	Output	Key skills
Data Engineer	Build and maintain data infrastructure	Reliable pipelines, clean datasets	SQL, Python, pipeline design, systems
Data Scientist	Extract insights and build models from data	ML models, statistical analysis	Python, ML, statistics, experimentation
Data Analyst	Explore data to answer business questions	Reports, dashboards, ad-hoc analysis	SQL, Excel, BI tools, business acumen

Data engineer's role in context:

Data engineers build the foundation — they ensure clean, reliable, timely data is available for data scientists and analysts. Without good data engineering, data scientists spend 80% of their time on data cleaning instead of modeling, and analysts cannot trust the numbers they see.

SECTION 2

SECTION 2 – Scenario-Based Questions (Q46 – Q65)

20 real-world scenarios testing your practical problem-solving ability — no tool-specific knowledge assumed.

Q46. Your nightly batch pipeline that processes 500 GB of data is taking 8 hours. The business needs it done in 2 hours. How do you approach this?**Answer — Systematic optimization approach:**

- Step 1 — Profile first: Identify which step in the pipeline is the actual bottleneck. Is it extraction, transformation, or loading? Measure each step before optimizing blindly.
 - Step 2 — Switch to incremental: If the pipeline does a full load every night, switch to incremental loading. Only process new or changed records since the last run. This typically reduces data volume by 80–95%.
 - Step 3 — Parallelize: Break the pipeline into independent chunks that can run simultaneously. For example, process each date partition or each region as a separate parallel task.
 - Step 4 — Optimize transformations: Identify expensive operations — Cartesian joins, full table scans, functions applied to filter columns. Rewrite them using partition-aware filtering.
-

- Step 5 — Push transforms to the destination: If transformations are happening outside the database, move them into the warehouse using SQL. Modern warehouses are highly optimized for set-based SQL operations.
- Step 6 — Optimize data formats: If reading CSV files, convert to columnar formats (Parquet, ORC). Columnar formats with compression reduce I/O by 5–10x.

Q47. Your pipeline ran successfully but the target table has duplicate rows after a retry. How do you debug and fix this?

Answer — Root cause investigation:

- Check the load pattern: Is the pipeline using INSERT (appends) or UPSERT/MERGE (update-or-insert)? If it uses INSERT without deduplication, every retry adds another copy.
- Check the retry logic: Did the pipeline mark the run as failed even though the load step completed? If the job fails after loading but before marking success, the next retry loads the data again.
- Check for concurrent runs: Were two instances of the pipeline running at the same time? Concurrent runs without locking can both insert the same batch.

Immediate fix:

- Deduplicate in the destination: Use `SELECT DISTINCT` or `ROW_NUMBER() OVER (PARTITION BY primary_key ORDER BY loaded_at DESC) = 1` to identify and remove duplicates.

Long-term fix:

- Change INSERT to UPSERT / MERGE on a unique business key.
- For partitioned tables, use partition overwrite — delete the partition then reload it atomically.
- Add a unique constraint on the primary key in the destination table to reject duplicates at the database level.

Q48. A critical business dashboard shows wrong revenue numbers. How do you investigate this as a data engineer?

Answer — Structured debugging approach:

- Step 1 — Check data freshness: Is the data stale? Verify `MAX(loaded_at)` on the revenue table. If the last pipeline run was yesterday, the dashboard is showing yesterday's numbers.
- Step 2 — Trace lineage: Which tables feed the revenue metric? Trace back through every transformation in the pipeline chain.
- Step 3 — Row count checks: Compare row counts between source and destination at each pipeline step. Where does the count diverge?
- Step 4 — Check for duplicates: `SELECT COUNT(*)` vs `SELECT COUNT(DISTINCT order_id)`. If these differ, duplicates are inflating the revenue.
- Step 5 — Spot-check specific records: Take 5 known transactions from the source system and verify they exist with correct values in the warehouse.
- Step 6 — Check recent changes: Was there a code deployment, schema change, or filter modification in the pipeline recently?

After fix:

Backfill affected date ranges, notify the business team, and add an automated reconciliation check that compares warehouse revenue to source system daily.

Q49. You are designing a pipeline that must ingest data from a source API that has a rate limit of 100 requests per minute. How do you handle this?

Answer:

- Throttle API calls: Implement rate limiting in the extraction code — add a deliberate delay between requests (e.g., max 80 requests/minute to stay safely under the limit with headroom).
- Exponential backoff with jitter: When a rate limit error (HTTP 429) is returned, wait before retrying — starting with a short wait and doubling each time with a small random jitter. This prevents thundering herd retries.
- Batch pagination: Fetch data in large pages (maximum allowed per request) to minimize total request count. If the API supports 1000 records per page, 1 million records = 1000 requests, not 1 million.
- Request queue with concurrency control: Instead of firing all requests in parallel, use a queue that processes requests at a controlled rate — respecting the rate limit even when the pipeline is parallelized.
- Caching: Cache reference or lookup data that rarely changes (e.g., product catalog, country codes). Avoid redundant API calls for static data.
- Incremental fetching: Only request new or changed records since the last run, not a full re-fetch every time.

Q50. Your pipeline reads data from a production database and is causing performance issues on the source. How do you fix this?

Answer:

- Use a read replica: Route all ETL queries to a dedicated read replica instead of the primary production database. Read replicas receive a copy of all data without affecting write traffic on the primary.
- Switch to CDC: Instead of polling the database with heavy queries, implement log-based CDC that reads the database's transaction log. This has near-zero impact on the source.
- Schedule during off-peak hours: Run heavy extractions during low-traffic windows (overnight) when production traffic is minimal.
- Limit concurrency: Configure the pipeline to use a small number of database connections (e.g., 3–5 max). Prevent the ETL from monopolizing the connection pool.
- Paginate queries: Replace large full-table queries with paginated incremental queries. Never `SELECT * FROM orders` in one query — use `WHERE id > last_id LIMIT 10000` in a loop.
- Add indexes: Ensure the columns used in ETL `WHERE` clauses (`updated_at`, `created_at`) have proper indexes on the source database to avoid full table scans.

Q51. You need to migrate a large dataset from one system to another with zero data loss. What is your migration plan?

Answer — Safe migration plan:

- Step 1 — Inventory and assess: Catalog all tables, row counts, schemas, and dependencies. Estimate data volume and migration time.
- Step 2 — Schema migration: Create equivalent schema in the destination. Handle data type differences and naming conventions.
- Step 3 — Historical data load: Export data to an intermediate format (Parquet, CSV) on cloud storage. Load from there into the destination using bulk load tools. This is faster and more reliable than direct DB-to-DB transfer.

- Step 4 — Validation: For every table, compare row count, column checksums, and sample records between source and destination. Do not proceed until validation passes.
- Step 5 — Dual-write period: Update pipelines to write to both old and new destinations simultaneously. Run for 2–4 weeks, comparing outputs to confirm the new system produces identical results.
- Step 6 — Cutover: Switch consumers to the new destination. Keep the old system active in read-only mode for 2–4 weeks as a fallback.
- Step 7 — Decommission: Once all consumers are on the new system and stable, decommission the old one.

Q52. A source system starts delivering data with a new column your pipeline does not expect. What happens and what do you do?

Answer:

What can happen depending on pipeline design:

- Strict schema validation: Pipeline fails with a 'unexpected column' or schema mismatch error. Good — you catch it immediately.
- No validation: The new column is silently ignored. You lose data you might need later.
- Typed destination table: INSERT fails because the destination table does not have this column.

Immediate response:

- Do not block: Store the raw data first (always preserve raw), then investigate the schema change.
- Assess the change: Is this a new optional column (additive) or a breaking change (renamed/removed column)?
- Additive column: Add the column as nullable to the destination table. Update the pipeline to pass through the new column.
- Breaking change: Create a new version of the destination table and migrate consumers gradually.

Prevention:

Implement schema change detection — a daily job that compares the current source schema to the expected schema and alerts when they diverge. This gives you advance warning before the pipeline breaks.

Q53. You need to build a pipeline for a dataset where some records can arrive up to 3 days late. How do you design for this?

Answer:

- Reprocessing window: Instead of treating each day as a closed batch after it runs, always reprocess the last N days (e.g., last 5 days) on every run. Late arrivals from up to 3 days ago will be caught.
- Partition by event time: Store records in partitions based on the event timestamp (when it happened), not the load time. A late record arriving today but timestamped 2 days ago lands in the correct historical partition.
- Idempotent partition overwrite: Since partitions are overwritten on each reprocessing run, late arrivals are naturally incorporated without creating duplicates.
- Provisional vs final flag: Mark records in rolling window partitions as 'provisional'. After the reprocessing window has passed (e.g., 5 days), mark the partition as 'final'. Consumers know not to use provisional data for definitive reporting.

- Alert on extreme late arrivals: If a record arrives more than 7 days late (beyond the reprocessing window), route it to a late_arrivals table for manual review.

Q54. Your pipeline has been running successfully for months and suddenly fails. How do you debug it?

Answer — Structured root cause analysis:

- Read the full error: Don't just look at the summary. Read the complete error message and stack trace — it almost always contains the specific cause.
- Check what changed recently: Look at code deployments in the last 24 hours. Has any pipeline code, configuration, or dependency been updated?
- Check the source system: Did the source schema change? Did the source return an unusual response (empty data, new format, null where not expected before)?
- Check infrastructure: Did a database connection time out? Did a storage bucket run out of space? Did an API credential expire?
- Check data anomalies: Did the source data volume spike? (10x more records than usual can cause memory or timeout failures). Did a previously non-null column start returning NULLs?
- Reproduce in isolation: Re-run the failing task with the same input data on a development environment to reproduce the error outside production.

After fixing:

Add a proactive alert for the root cause so this class of failure is caught before it breaks the pipeline next time.

Q55. You are asked to implement data quality checks in an existing production pipeline without disrupting it. How do you approach this?

Answer — Phased implementation:

- Phase 1 — Audit mode: Add quality checks that LOG results without blocking the pipeline. For 2 weeks, observe what the current data quality actually is without breaking anything.
- Phase 2 — Baseline: After 2 weeks, you have a baseline. You know which checks always pass (safe to make blocking) and which fail occasionally (need investigation first).
- Phase 3 — Blocking critical checks: Promote the most important, always-passing checks to blocking mode. These cause the pipeline to fail and alert if the check fails.
- Phase 4 — Alert on warning checks: For checks that occasionally fail, keep them non-blocking but trigger alerts so engineers are aware without stopping the pipeline.

Key checks to add first:

- Row count: Destination row count must be within 5% of source row count.
- Not null: Critical columns (order_id, customer_id) must have zero NULLs.
- Uniqueness: Primary key column must have no duplicates.
- Freshness: Data must be loaded within the SLA window.

Q56. How would you design a pipeline that needs to process data from 50 different sources, each with a slightly different schema?

Answer — Metadata-driven pipeline design:

- Common schema mapping layer: Define a canonical target schema. For each source, store a mapping in a config table: 'source_column X maps to target_column Y, apply transformation Z.'

- Metadata-driven execution: The pipeline reads the mapping config at runtime and applies the correct transformations per source — no hardcoded source-specific logic in the pipeline code itself.
- Standardized raw landing: All sources land in a raw zone in their original format. Transformation happens in a separate step, not during ingestion.
- Source-specific validation: Each source has its own validation rules stored in config. The pipeline loads and applies them dynamically.
- Onboarding template: Adding a new source requires only a config entry — no code changes. This makes the pipeline scalable to 500 sources with the same codebase.

Key principle:

Never write source-specific if/else logic in pipeline code. Configuration should drive behavior. Code should only know 'read config, apply mappings, validate, load.'

Q57. Your pipeline loads data into a partitioned table and occasionally a partition ends up partially loaded. How do you prevent this?

Answer:

Root causes of partial partitions:

- The pipeline fails halfway through loading a partition, leaving partial data.
- The pipeline writes records incrementally to the partition — consumers see partial data before the load completes.

Solution — Atomic partition swap pattern:

- Step 1: Load all new data into a staging partition (e.g., partition labeled 'staging_20240601').
- Step 2: Validate the staging partition — row counts, quality checks, reconciliation.
- Step 3: Only after validation passes, atomically swap the staging partition to the final partition name.
- Step 4: If any step fails, the staging partition is simply dropped. The final partition is never touched.

Alternative — Write-then-rename file pattern:

For file-based systems, write to a temp file path, then atomically rename/move to the final path. Consumers only see the final path, never the in-progress temp file.

Key principle:

Consumers should never see a partially-written state. Use atomic operations (swap, rename, overwrite) as the final step.

Q58. You are the only data engineer at a startup. How do you build a data platform from scratch with limited resources?

Answer — Pragmatic minimal viable data platform:

- Phase 1 (Week 1–2): Pick managed, SaaS-first tools. Use a managed ELT ingestion tool to sync from your source databases without writing custom extraction code. Use a cloud data warehouse (Snowflake, BigQuery, or Redshift Serverless) — no infra to manage.
- Phase 2 (Week 3–4): Add SQL transformations on top of the raw data. Build cleaned, modeled tables that business users can query.
- Phase 3 (Month 2): Add a BI tool. Connect it to the curated tables. Give analysts access without routing through you for every question.

- Phase 4 (Month 3): Add basic data quality checks and freshness monitoring. Set up alerts for pipeline failures and SLA breaches.
- Phase 5 (Month 4+): Add custom pipelines only when the managed tools hit limitations or when the cost/benefit justifies it.

Key principle:

At a startup, your value is in data models and business insights — not in maintaining infrastructure. Buy the commodity, build the differentiator. Add complexity only when proven necessary.

Q59. A new business requirement asks you to backfill 2 years of historical data after fixing a transformation bug. How do you do this safely?

Answer — Safe historical backfill strategy:

- Step 1 — Identify affected scope: Determine exactly which date range was affected by the bug. Check when the bug was introduced vs. when it was fixed.
- Step 2 — Preserve current data: Before backfilling, take a snapshot or backup of the current state of the affected tables. If something goes wrong during backfill, you can restore.
- Step 3 — Use idempotent re-runs: Trigger the pipeline for each affected date partition. Idempotent pipelines will delete-and-reload each partition, producing correct results.
- Step 4 — Parallelize the backfill: Process multiple date partitions in parallel (not sequentially) to complete the 2-year backfill faster.
- Step 5 — Validate after each batch: After each batch of partitions is backfilled, run reconciliation checks before proceeding to the next batch.
- Step 6 — Notify consumers: Inform downstream teams that data is being corrected and provide a timeline. They should not use affected data for critical decisions during backfill.

Common mistake:

Running backfill on the production pipeline at full speed and overloading the source system or the warehouse. Rate-limit the backfill to run at a controlled pace.

Q60. How would you handle a scenario where two pipelines accidentally process and load the same data simultaneously?

Answer:

Prevention — avoid simultaneous runs:

- Use job locking: Before a pipeline run starts, check if another instance is already running (e.g., check a 'pipeline_status' table). If already running, abort the new instance.
- Set concurrency limits: Configure your orchestration tool to allow maximum one active run per pipeline at a time.
- Idempotent design: Even if two runs overlap, an idempotent pipeline (using UPSERT or partition overwrite) produces correct results — duplicates are prevented by the write strategy itself.

Recovery if it already happened:

- Identify duplicate records using `ROW_NUMBER() OVER (PARTITION BY primary_key ORDER BY loaded_at) > 1`.
 - Delete duplicates keeping only one copy per primary key.
 - Add a unique constraint on the primary key to prevent this at the database level in the future.
-

Q61. Your data pipeline SLA is that data must be available by 8 AM daily. What monitoring would you put in place to protect this?

Answer — SLA monitoring strategy:

- Pre-run dependency check (6 AM): Verify all upstream data sources are available and complete before the pipeline starts. If an upstream table is not ready, alert immediately rather than failing at 7:55 AM.
- Pipeline duration tracking: Track how long each pipeline step takes historically. If today's run is already 30% longer than the 30-day average at 7 AM, trigger an early warning alert.
- Progress checkpoints: For long-running pipelines, add intermediate checkpoints — 'by 6:30 AM, extraction should be done; by 7:00 AM, transformation should be done.'
- Final SLA check: At 7:50 AM, run an automated check: Is the data loaded? Is MAX(loaded_at) fresh? If not, trigger an urgent alert to the on-call engineer.
- Escalation policy: Define what happens if SLA is missed — who is notified, what interim data (yesterday's data with a 'stale' flag) should be shown on dashboards.

Monitoring principle:

Don't just alert when things fail — alert early when things are trending toward failure. A slow pipeline at 6 AM is easier to recover than a failed pipeline at 7:58 AM.

Q62. How do you design a data pipeline to support GDPR's right to erasure (right to be forgotten)?

Answer:

- Step 1 — Data discovery: Use your data catalog to identify every table across every pipeline that contains PII for a given user. This is why tagging PII columns at ingestion time is critical.
- Step 2 — Deletion pipeline: Build a deletion pipeline triggered by a deletion request. It iterates through all identified tables and either hard-deletes the rows or replaces PII values with NULLs or anonymous tokens.
- Step 3 — Cascading deletes: Ensure deletion cascades to all derived tables built from the raw data (aggregated tables, warehouse models, data lake files).
- Step 4 — Audit log: Record each deletion request — when it was received, what data was deleted, and confirmation that deletion is complete. This is required for GDPR compliance proof.
- Step 5 — Handle archives and backups: Deletion must extend to backups and archives. This is the hardest part — many organizations handle this by using tokenization so archived backups only contain tokens, not real PII.

Design principle:

Build deletion capability into the platform from day one. Retrofitting deletion onto a complex data platform with many tables is extremely difficult and error-prone.

Q63. You have a pipeline where the transformation logic is very complex and takes many hours to explain to new team members. How do you improve this?

Answer:

- Modularize transformations: Break one massive transformation into a sequence of small, named, single-purpose steps. Each step should be explainable in one sentence: 'This step removes cancelled orders.' 'This step calculates customer lifetime value.'
- Add inline documentation: Write comments explaining why each transformation exists, not just what it does. 'We exclude test users (user_id starts with 'TEST_') because they inflate conversion rates.'

- Build intermediate tables: Instead of one giant transformation with 50 joins, create intermediate named tables at logical checkpoints. Each intermediate table is also inspectable and testable independently.
- Write transformation tests: Tests document expected behavior. A test called 'test_cancelled_orders_are_excluded' is more informative than a code comment.
- Create a data dictionary: Document each column in the final output — what it means, how it is calculated, what edge cases are handled.

Key principle:

Code that is hard to explain is usually also hard to test and hard to maintain. Complexity is a risk — simplify proactively, not just when things break.

Q64. A downstream team reports that your pipeline's data does not match their expectations. How do you resolve this disagreement?

Answer:

- Start with facts, not opinions: Pull specific example records where the disagreement exists. Compare values side by side — source, your pipeline's output, and what they expect.
 - Check definitions: 'Revenue' might mean gross revenue to you and net revenue (after refunds) to them. Many data disagreements are actually definition disagreements, not pipeline bugs.
 - Trace lineage together: Walk through the pipeline transformation steps together. Show them every filter, join, and aggregation applied to explain where the numbers come from.
 - Document agreed definitions: Once you identify the discrepancy, write down the agreed business definition and update the data catalog so the same confusion does not happen again.
 - If they are right: Fix the pipeline, backfill affected history, and add a test to prevent regression.
 - If you are right: Update your documentation to explain the logic more clearly, and help them build the correct query on top of your data for their use case.
-

Q65. How would you design a self-healing data pipeline that recovers automatically from common failures?

Answer — Self-healing pipeline design principles:

- Automatic retries with backoff: Configure each task to retry on failure with exponential backoff. Transient failures (network timeouts, temporary service unavailability) resolve themselves on retry.
 - Idempotent tasks: Every task must be safe to retry. This is the foundation — without idempotency, automatic retries cause more harm than good.
 - Dependency-aware failure handling: If an upstream task fails, mark downstream tasks as 'skipped' or 'blocked' rather than letting them run on incomplete data.
 - Dead letter queue for bad records: Route records that fail processing to a DLQ instead of crashing the pipeline. The pipeline continues processing good records. Bad records are reviewed and replayed after fixing the root cause.
 - Alerting with context: When a pipeline finally fails after exhausting retries, the alert should include the full error context, affected date range, and a link to logs — not just 'pipeline failed.'
 - Automated backfill trigger: For pipelines with clear SLAs, if a run is missed, automatically trigger a backfill run for the missed window.
-

— SECTION 3 —

SECTION 3 – Best Practices & Architecture (Q66 – Q75)

10 questions on how to architect and operate production-grade data pipelines using good engineering principles.

Q66. What is the Medallion Architecture (Bronze / Silver / Gold) and how do you implement it?

Answer:

Medallion Architecture organizes a data lake or data platform into three progressive quality layers, each representing a higher level of data refinement.

Layer	Also called	Contents	Consumers
Bronze	Raw / Landing	Exact copy of source data. No transformations. Immutable.	Data engineers only
Silver	Cleaned / Enriched	Validated, deduplicated, type-cast, standardized data.	Data engineers, data scientists
Gold	Business / Serving	Aggregated, business-logic applied, KPIs, ready for reporting.	Analysts, BI tools, ML models

Key principle:

Never transform in place. Each layer is built from the layer below it. If transformation logic is wrong, re-process from Bronze without re-extracting from the source. Bronze is your audit trail and safety net — once raw data is in Bronze, it is never lost.

Q67. What is pipeline observability and what does a production-grade monitoring strategy include?

Answer:

Pipeline observability is the ability to understand the current health and historical behavior of your data pipelines through metrics, logs, and alerts — without needing to manually inspect each run.

A complete observability strategy covers:

- **Logging:** Structured logs (JSON format) for every pipeline run — start time, end time, rows processed, rows rejected, errors encountered. Ship to a centralized log system.
- **Metrics:** Track pipeline-level metrics over time — run duration, row counts, error rates, data freshness, SLA compliance rate. Alert on anomalies.
- **Alerting:** Alert on: pipeline failure, SLA breach, row count anomaly (>20% unexpected change), data freshness breach, schema change detected.
- **Health dashboard:** A single-pane view of all pipelines — last run status, last run time, duration trend, data quality score.
- **Lineage and impact:** When a pipeline fails, know instantly which dashboards, models, and downstream consumers are affected.

Q68. What are the best practices for managing secrets and credentials in data pipelines?

Answer:

- Never hardcode credentials: No passwords, API keys, or tokens in code, config files, or version control — ever. A single committed secret can cause a major security breach.
- Use a secrets manager: Store all credentials in a dedicated secrets management system. Inject them at runtime when the pipeline executes, not at deploy time.
- Environment-based injection: Pass credentials as environment variables at runtime. The pipeline code reads from environment variables — not from files or hardcoded strings.
- Least privilege: Each pipeline service account should have only the minimum permissions it needs. An ETL job that only reads from one table should not have admin credentials.
- Credential rotation: Rotate credentials regularly. Use short-lived credentials (tokens that expire in hours, not years) wherever possible.
- Audit access: Log every time a secret is accessed. Alert on unexpected access patterns — for example, a pipeline suddenly accessing a secret it has never used before.

Q69. How do you version and manage data pipeline code in a team environment?

Answer:

- Everything in version control: All pipeline code, SQL queries, config files, schema definitions — in Git. No live editing on production servers. No undocumented manual changes.
- CI/CD pipeline: Every change goes through: code review (PR) → automated tests (unit tests, integration tests) → staging deployment → production deployment. No direct pushes to production.
- Environment separation: Maintain separate dev, staging, and production environments with their own databases and storage. Never test on production data.
- Pipeline versioning: When making breaking changes to a pipeline's output schema, create a new version (e.g., pipeline_orders_v2) that runs alongside the old version during a migration period.
- Infrastructure as code: All pipeline infrastructure (servers, storage buckets, databases, access controls) defined in code (Terraform, CloudFormation) — no manual provisioning.
- Change log: Maintain a CHANGELOG documenting what changed, when, and why. Invaluable for debugging regressions.

Q70. What is the difference between building vs buying data pipeline tools, and how do you decide?

Answer:

Aspect	Build (Custom Code)	Buy (SaaS / Managed Tool)
Time to value	Slow — weeks to months to build	Fast — hours to days to set up
Initial cost	High engineering time	Low setup, ongoing subscription fee
Flexibility	Fully custom to your exact needs	Limited to vendor's feature set
Maintenance	Your team maintains it forever	Vendor handles maintenance and upgrades

Aspect	Build (Custom Code)	Buy (SaaS / Managed Tool)
Vendor lock-in	None	High — migration is painful

Decision framework:

- Buy for commodity capabilities: ingestion from standard sources, basic orchestration, BI dashboards. These are solved problems — don't reinvent them.
- Build for competitive differentiation: proprietary transformation logic, custom algorithms, unique business rules. This is where your value lies.

Rule of thumb:

If you can describe your need in a standard product category (ETL tool, BI tool, orchestration tool), buy it. If your need is unique to your business, build it.

Q71. How do you design a data pipeline for maintainability over the long term?**Answer:**

- Modularity: Break pipelines into small, single-responsibility tasks. Each task does one thing and does it well. This makes it easy to understand, test, and replace individual parts.
- No magic numbers: Every threshold, filter value, or configuration should be in a named constant or config file — not hardcoded as a mysterious number in the middle of code.
- Self-documenting code: Name variables, functions, and tasks clearly. `clean_customer_records()` tells you more than `process_step_3()`.
- Automated tests: Write tests for transformation logic. Tests serve as living documentation of what the code is supposed to do.
- Dependency management: Declare all dependencies explicitly. No implicit coupling between pipelines. If Pipeline B needs Pipeline A's output, that dependency is declared, not assumed.
- Regular refactoring: Treat pipeline code like production application code — schedule time to clean up technical debt before it accumulates into an unmaintainable codebase.

Key metric:

How long does it take a new team member to understand a pipeline well enough to make a safe change? The answer should be hours, not weeks.

Q72. What strategies do you use to handle failures in production data pipelines?**Answer:**

- Automatic retries: Configure tasks to retry 2–3 times with exponential backoff before marking as failed. Most transient failures (network timeouts, temporary locks) resolve on retry.
- Partial failure isolation: A failure in one pipeline task should not automatically cascade and cancel all unrelated tasks. Isolate failures to affected branches of the dependency graph.
- Alerting with context: Failure alerts must include the error message, the affected date range, the task that failed, and a link to the full log. Not just 'pipeline failed.'
- On-call rotation: Have a defined on-call process — who owns each pipeline, who to page when it fails at 3 AM, and what the escalation path is.
- Runbooks: Document the response procedure for common failures — 'if you see error X, do Y.' This allows junior engineers to handle incidents independently.
- Post-mortems: For significant failures, conduct a blameless post-mortem — what happened, why, what was the impact, what is the fix, and how do we prevent recurrence.

Q73. What is a Lambda Architecture vs a Kappa Architecture, and when would you use each?**Answer:**

Aspect	Lambda Architecture	Kappa Architecture
Layers	Batch layer + Speed layer + Serving layer	Single streaming layer + Serving layer
Complexity	High — two separate codebases for same logic	Lower — one unified processing codebase
Historical reprocessing	Batch layer handles it natively	Replay historical events through the streaming job
Use when	You need both accurate historical and real-time results	Your team is comfortable with streaming-first design

Explanation:

Lambda was the dominant architecture before unified streaming engines matured. It uses batch processing for accurate historical results and a streaming layer for low-latency recent results, then merges them in a serving layer. Kappa simplifies by using only a streaming layer for everything — historical reprocessing is done by replaying all historical events through the stream processor. Lambda is more proven; Kappa is simpler to maintain but requires strong streaming expertise.

Q74. How do you handle PII data in a data pipeline to meet compliance requirements?**Answer:**

- Identify and tag PII at ingestion: Every column containing PII (name, email, phone, IP address, SSN) must be tagged in the data catalog at the point of ingestion. You cannot protect what you cannot find.
- Minimize collection: Only collect PII that is strictly necessary. If a column is not needed for the pipeline's purpose, do not ingest it.
- Pseudonymization: Replace direct PII with consistent, reversible tokens. Analysts work with tokens for joins and aggregations. Only authorized services can look up the real value.
- Anonymization for analytics: For data used in reporting, ML training, or shared externally, replace PII with anonymized values that cannot be reversed.
- Access control: Enforce role-based access — only authorized roles can query raw PII columns. Apply column-level masking policies in the warehouse.
- Encryption: PII must be encrypted at rest (storage encryption) and in transit (TLS). Do not store PII in unencrypted log files or pipeline monitoring systems.
- Deletion pipeline: Build automated deletion capability. GDPR right to erasure must be fulfilled within 30 days of request.

Q75. How do you design a data pipeline that is easy to test?**Answer:**

- Pure transformation functions: Write transformation logic as pure functions — input data in, output data out, no side effects. Pure functions are trivially easy to unit test.
- Separate concerns: Keep extraction (reading from source), transformation (business logic), and loading (writing to destination) in separate, independently testable modules.

- Test with known inputs and expected outputs: For each transformation, define a small test dataset (5–10 rows) with known input and a verified expected output. Run this test in CI on every code change.
 - Mock external dependencies: In unit tests, replace real database connections and API calls with mock objects. Tests should run without any external services.
 - Integration tests: In staging, run end-to-end tests with a sample of real data to verify the full pipeline behaves correctly.
 - Data quality tests: Add automated assertions on the output — row count within expected range, no NULLs in key columns, uniqueness on primary key. These run as part of the pipeline itself.
-

— SECTION 4 —

SECTION 4 – Performance Optimization (Q76 – Q85)

10 questions on making data pipelines faster, cheaper, and more efficient — without assuming any specific processing tool.

Q76. What are the most effective techniques for optimizing a slow SQL query in a data warehouse?

Answer — Query optimization checklist:

- Use EXPLAIN / query profile: Before changing anything, look at the execution plan. Find the most expensive step — is it a full table scan, a large join, or a sort?
 - Filter early and aggressively: Apply WHERE clauses as early as possible to reduce row counts before joins and aggregations. Never join 1 billion rows when a filter reduces it to 1 million first.
 - Avoid functions on filter columns: WHERE YEAR(order_date) = 2024 prevents index/partition use. Rewrite as: WHERE order_date >= '2024-01-01' AND order_date < '2025-01-01'.
 - Select only needed columns: Avoid SELECT *. Columnar databases read each column separately — selecting fewer columns reads less data.
 - Partition pruning: Ensure filters on the partition column are present so the engine skips irrelevant partitions entirely.
 - Pre-aggregate before joining: If joining a large table with aggregations, aggregate first, then join. Joining 10 rows is faster than joining 10 million.
 - Use CTEs for clarity and optimization: Break complex queries into named CTEs. This also helps the optimizer understand the intent of each step.
-

Q77. How do you optimize incremental pipelines to run as fast as possible?

Answer:

- Precise watermark windows: Fetch only records changed in the minimum necessary window. If your pipeline runs hourly, fetch only the last 1 hour plus a small overlap buffer for late arrivals — not the last 24 hours every run.
 - Index the watermark column: The source database should have an index on the column used for incremental filtering (updated_at, created_at). Without an index, the source does a full table scan on every pipeline run.
-

- Avoid reading deleted records repeatedly: If your incremental logic reads all records updated in a window, and that window includes many records updated long ago but never re-processed, use a more precise query that limits to the exact new window.
- Parallelize partitions: Process multiple source shards or date ranges in parallel instead of sequentially.
- Minimize re-transformation: If only a small subset of records changed, do not re-transform the entire dataset. Apply transformations only to the incremental records, then merge with the existing clean data.
- Optimize destination writes: Bulk inserts are far faster than row-by-row inserts. Always use bulk load / COPY operations when loading large volumes.

Q78. What is predicate pushdown and how does it improve pipeline performance?

Answer:

Predicate pushdown means moving filter conditions (WHERE clauses) as close to the data source as possible — ideally evaluating them at the storage layer before data is even read into memory.

Without predicate pushdown:

Read all 1 billion rows from storage into memory, then apply the filter. Reads 100% of data even if the filter selects 0.1%.

With predicate pushdown:

The storage layer evaluates the filter first and returns only matching rows. The processing layer receives a tiny fraction of the total data.

Practical example:

Query: `SELECT * FROM events WHERE country = 'IN' AND date = '2024-06-01'`. With predicate pushdown, the storage layer reads only the `date=2024-06-01` partition and skips row groups where country is never 'IN'. Result: 100x less data read.

How to enable it:

Use columnar formats (Parquet, ORC) that support row group statistics. Avoid wrapping filter columns in functions (which prevent the storage layer from evaluating the predicate). Use partitioned storage aligned with your most common filter columns.

Q79. What are small file problems in a data lake and how do you solve them?

Answer:

Small file problem: when a data lake accumulates millions of tiny files (kilobytes each) instead of fewer large files (hundreds of megabytes each), query and pipeline performance degrades severely.

Why small files are bad:

- Each file requires a separate filesystem metadata lookup, network round-trip, and I/O operation. Millions of files mean millions of overhead operations.
- Query engines have a per-file overhead — reading 10,000 files of 10KB each is far slower than reading 10 files of 10MB each, even though the total data is the same.

Root causes:

- High-frequency micro-batch writes that generate one file per batch.
- Over-partitioning: partitioning by both date AND hour AND user_id creates billions of tiny partitions.
- Frequent small appends without compaction.

Solutions:

- **Compaction jobs:** Regularly run a job that reads many small files and rewrites them as fewer large files (target 100–500 MB per file).
- **Coarser partitioning:** Partition by date instead of date+hour+user_id.
- **Buffer before writing:** Accumulate data in memory or a staging area before writing to the lake, ensuring each write produces a reasonably-sized file.

Q80. How do you reduce the cost of running data pipelines in the cloud?

Answer:

- **Right-size compute:** Don't use the largest instance type by default. Profile actual CPU and memory usage and choose the smallest instance that comfortably handles the workload.
- **Spot / preemptible instances:** For batch pipelines that can tolerate interruption and restart, use spot/preemptible instances (60–90% cheaper than on-demand). Design pipelines to checkpoint and resume.
- **Optimize data format and compression:** Replace CSV/JSON with Parquet + Snappy/ZSTD. Typically 5–10x smaller, which reduces both storage and data transfer costs.
- **Storage tiering:** Move infrequently accessed data to cheaper storage tiers (warm/cold/archive). Historical data older than 6 months may only be queried once per quarter.
- **Delete unused data:** Identify tables and files that are never queried. Remove them. Many data lakes accumulate enormous amounts of forgotten data.
- **Incremental vs full load:** Running incremental pipelines processes 10–100x less data than full loads — the cost savings are proportional.
- **Schedule wisely:** Some cloud services (managed query engines) have minimum billing per query. Batch multiple small queries into fewer larger queries to reduce per-query overhead.

Q81. What is data compression and which compression strategies are best for different scenarios?

Answer:

Data compression reduces the physical size of stored data by encoding it more efficiently, reducing storage cost and I/O time when reading or transferring data.

Codec	Compression ratio	Speed	Best for
Snappy	Medium	Very fast	Analytical data lakes — balance of speed and size
ZSTD	High	Fast	High compression without much speed penalty
GZIP	High	Slower	Cold archival data — size matters more than speed
LZ4	Low-medium	Fastest	Real-time processing where latency is critical
Uncompressed	None	Fastest read	Rarely — only for tiny datasets

Column-level compression (columnar formats):

In columnar formats (Parquet, ORC), each column is also compressed independently using encoding strategies — run-length encoding for repeated values, dictionary encoding for low-cardinality columns. A country column with 50 unique values in 1 billion rows compresses to essentially just 50 values + a lookup index.

Q82. What are the best strategies for optimizing large JOIN operations in data pipelines?

Answer:

- Filter before joining: Apply WHERE conditions to both sides of the join before joining. Never join 1 billion rows against 500 million rows if filters reduce them to 10 million and 1 million first.
 - Join on indexed columns: Ensure join columns have indexes in the source database (for SQL-based pipelines). Without indexes, joins degrade to full table scans on every join.
 - Use the right join type: Use INNER JOIN when you only want matching rows. Using LEFT JOIN when INNER JOIN is correct forces the engine to process and handle non-matching rows unnecessarily.
 - Broadcast small tables: When joining a large table with a small lookup table (e.g., country codes, product categories), consider materializing the small table in memory on all workers — eliminates expensive data shuffling.
 - Pre-aggregate before joining: If you need SUM(revenue) per customer joined to customer demographics, aggregate first (reduce to one row per customer), then join. Never join at the raw transaction level when aggregated results are sufficient.
 - Avoid joining on NULLs: NULL = NULL is always false in SQL — joining on a column with many NULLs produces unexpected results and forces the optimizer to handle edge cases.
-

Q83. How do you handle a pipeline that processes a dataset that grows significantly over time (from 1TB to 100TB)?

Answer:

- Design for horizontal scale from day one: Build pipelines that can process data in parallel chunks — by date, by region, by source shard. Adding more workers increases throughput proportionally without changing the pipeline logic.
 - Incremental processing is non-negotiable at scale: What works as a nightly full load at 1TB becomes unworkable at 100TB. Incremental pipelines scale because each run only processes the small delta, regardless of total historical volume.
 - Revisit partitioning strategy: A daily partition that works at 1TB may produce excessively large files at 100TB. Re-partition to a finer granularity (hourly partitions) as data volume grows.
 - Data lifecycle management: Aggregate and archive old data. At 100TB, keeping 5 years of raw granular events is expensive. Summarize events older than 90 days and archive raw data to cold storage.
 - Monitor and re-tune regularly: Pipeline configurations optimized for 1TB are often wrong for 100TB. Review job performance quarterly as volume grows.
 - Avoid single points of bottleneck: Ensure no single sequential step processes all 100TB. Every bottleneck at 1TB becomes a crisis at 100TB.
-

Q84. What is the small table vs large table join strategy and how does it affect performance?

Answer:

Two main strategies based on table sizes:

Strategy 1 — Broadcast join (small table replicated everywhere):

When one table is small (e.g., a 500KB country_codes lookup table), replicate it to every worker node. Each worker can join its portion of the large table locally without any data movement. Zero network shuffle. Extremely fast.

Strategy 2 — Shuffle/sort-merge join (both tables large):

When both tables are large, sort both tables by the join key and merge them. This requires shuffling data across nodes but handles any table size. Expensive due to network transfer but necessary when both sides are too large to broadcast.

How to decide:

If one table has fewer than ~100MB and fits in memory: broadcast it. If both tables are large: sort-merge join with proper partitioning on the join key to minimize shuffle volume.

Q85. How do you optimize data pipeline parallelism without overwhelming the destination system?

Answer:

Parallelism improves throughput but can crash or overload downstream systems (databases, APIs, warehouses) if unconstrained.

Strategies to balance parallelism and stability:

- Concurrency limits: Set a maximum number of parallel tasks that write to the destination simultaneously. Start conservatively (e.g., 5 parallel writers) and increase based on observed destination performance.
- Connection pooling: All parallel tasks share a bounded connection pool to the destination. This limits the total connections regardless of task count.
- Partition-level isolation: Each parallel task writes to a different partition. This avoids lock contention — tasks never compete to write to the same rows.
- Bulk load over row-by-row inserts: Use COPY or bulk insert operations instead of individual INSERT statements. One bulk load is orders of magnitude faster and generates far less destination load.
- Write to staging first: Parallel tasks write to independent staging tables. A single sequential step merges staging into the final destination. This eliminates write contention entirely.
- Monitor destination metrics: Watch destination CPU, connection count, and queue depth as you increase parallelism. Back off immediately if metrics spike.

SECTION 5

SECTION 5 – Advanced & Tricky Interview Questions (Q86 – Q100)

15 advanced questions that separate senior data engineers from juniors — edge cases, deep reasoning, and system thinking.

Q86. If you have an idempotent pipeline but you still see duplicates after a retry, what went wrong?

Answer:

This is a tricky question designed to test whether you truly understand idempotency. Many candidates claim their pipeline is idempotent without verifying it at every step.

Hidden causes of duplicates in supposedly idempotent pipelines:

- One step is not idempotent: Idempotency must hold at every step independently. The pipeline is only as idempotent as its least idempotent step. If the load step uses INSERT instead of MERGE, it appends on every retry.
- The deduplication key is wrong: UPSERT keyed on the wrong column — two different records share the same key value, causing one to be lost, while retries insert the other twice.
- Concurrent runs broke the assumption: Two pipeline instances ran simultaneously — both thought they were the first run and both inserted the same data.
- The extract step is not deterministic: The query used to extract data (e.g., based on a timestamp window) returns slightly different records on retry due to timezone handling or clock drift.

Debugging approach:

Audit every stage of the pipeline independently. Test each step in isolation by running it twice with the same input and verifying the output is identical.

Q87. What is the difference between TRUNCATE and DELETE in ETL pipelines and which is safer?**Answer:**

Aspect	TRUNCATE	DELETE
Speed	Very fast — deallocates entire table at once	Slow — logs every row removed individually
Rollback	Cannot rollback in most databases (DDL operation)	Can rollback — it is a DML transaction
WHERE clause	Not supported — removes ALL rows unconditionally	Supports WHERE — can remove specific rows
Risk	High — if something fails after truncate, table is empty	Lower — delete can be rolled back if in transaction

Safer pattern in ETL:

Do not TRUNCATE the production table directly. Instead: (1) Load new data into a staging table. (2) Validate the staging data. (3) Only after validation passes, atomically swap staging to production (RENAME or partition swap). If anything fails before step 3, the production table is never touched. This is far safer than TRUNCATE + INSERT in sequence.

Q88. Can a data pipeline be both real-time and accurate? What is the fundamental trade-off?**Answer:**

This is a deep architectural question. The short answer is: real-time and perfectly accurate are often in tension.

The fundamental trade-off — latency vs completeness:

- Real-time pipelines process data as it arrives. But events can arrive late (delayed by network, mobile sync, retries). A real-time result computed at 10:00 AM may be missing events that arrive at 10:15 AM but are timestamped at 9:55 AM.
- Batch pipelines wait for a window to close before processing. By waiting, they capture most late arrivals and produce more complete results — but with higher latency.

How production systems handle this:

- Two-layer approach: A real-time layer produces fast but provisional results (shown with a 'provisional' label). A batch correction layer runs hourly to incorporate late arrivals and publish final, accurate results.
- Configurable lateness tolerance: Accept events up to N minutes late before finalizing a window. Longer N = more accurate, higher latency. Shorter N = less accurate, lower latency.

Business question:

How late can results be? How wrong can they be? The answers set the design parameters. There is no universally correct answer — it is always a business trade-off.

Q89. Why does storing data partitioned by load date instead of event date cause reporting problems?

Answer:

This is a subtle but common data engineering mistake with significant downstream impact.

The problem:

If a table is partitioned by `load_date` (when the pipeline ran), and late-arriving events from yesterday are loaded today, those events land in today's partition — not yesterday's. A report querying yesterday's partition misses them entirely.

Example:

A user made a purchase on June 1st but the event arrived and was processed on June 3rd due to a mobile app sync delay. With `load_date` partitioning, this purchase appears in the June 3rd partition. A daily revenue report for June 1st will always undercount revenue, even after re-runs.

Correct approach:

Always partition by `event_date` (when the event actually happened). Store both `event_date` and `loaded_at` in every row. Use `event_date` for all business reporting. Use `loaded_at` only for pipeline debugging and monitoring.

Additional benefit:

With `event_date` partitioning, re-running the pipeline to incorporate late arrivals automatically updates the correct historical partitions without affecting other dates.

Q90. What happens to your incremental pipeline if the source system's `updated_at` column is not reliably maintained?

Answer:

This is a real production problem that breaks most standard incremental loading strategies.

What can go wrong with unreliable `updated_at`:

- `updated_at` not updated on all changes: Some code paths in the source application update records without touching `updated_at`. These changes are invisible to incremental pipelines forever.
- `updated_at` set to NULL: Records with NULL `updated_at` are never picked up by a watermark-based query.

- `updated_at` uses server time inconsistently: Records written during a daylight saving time transition or from servers in different timezones get incorrect timestamps.

Detection:

Run periodic reconciliation (weekly full count comparison) between source and destination. A growing discrepancy indicates missed records.

Solutions:

- Push for source fix: Request the source team to reliably maintain `updated_at` on all code paths.
- Use CDC: Log-based CDC captures every change regardless of application-level timestamp reliability.
- Hybrid approach: Incremental load daily for efficiency + weekly full reconciliation to catch any missed records.

Q91. How would you detect data drift in a production data pipeline without an ML model?

Answer:

Data drift is a change in the statistical properties of data over time. Without catching it, pipelines continue processing data that looks correct structurally but is statistically anomalous.

Detection techniques without ML:

- Row count monitoring: Track daily row count per table. Alert if today's count is more than 2 standard deviations from the 30-day average. A sudden 50% drop or 200% spike signals a problem.
- Column null rates: Track the percentage of NULLs per column daily. A column that was 0% NULL for months suddenly becoming 30% NULL indicates a source system change.
- Value distribution monitoring: For low-cardinality columns (status, country, category), track the percentage distribution of each value. Alert when a value's percentage changes dramatically.
- Min / Max / Mean tracking: For numeric columns, track daily statistics. Revenue that was averaging \$50 suddenly averaging \$500 needs investigation.
- New or missing values: Alert when a categorical column contains a new value that was never seen before (new category introduced upstream) or when an expected value disappears (potential data loss).

Implementation:

Run a daily 'data profiling' pipeline that computes these statistics and compares to a rolling historical baseline. Store results in a monitoring table and alert on anomalies.

Q92. What is the N+1 query problem in data pipelines and how do you solve it?

Answer:

The N+1 problem occurs when a pipeline fetches N records in one query, then makes an additional individual query for each of those N records to retrieve related data — resulting in N+1 total queries instead of 1 or 2.

Example:

You fetch 10,000 orders from the database (1 query), then for each order, make a separate query to look up the customer details (10,000 queries). Total: 10,001 queries. With 100ms per query, this takes 16 minutes instead of the 1 second a single JOIN would take.

Why it happens:

Developers write row-by-row processing logic (a for loop calling the database per iteration) instead of set-based operations. Very common when pipeline code is written by developers more familiar with application code than data engineering.

Solutions:

- JOIN at the database: Fetch all related data in a single query using SQL JOINS. The database is designed to do this efficiently.
- Batch lookup: Collect all IDs first (e.g., all order customer_ids), fetch all related records in one query (WHERE customer_id IN (...)), then join in memory.
- Cache lookups: Cache small, frequently used reference data in memory. Serve all N lookups from cache with zero additional queries.

Q93. What is the difference between eventual consistency and strong consistency, and which does a data warehouse provide?**Answer:**

Consistency Model	Definition	Example
Strong consistency	After a write completes, all readers immediately see the new value	Relational databases with ACID — once committed, every query sees it
Eventual consistency	After a write, readers will eventually see the new value — but not necessarily immediately	Distributed NoSQL systems — a write on one node propagates to others over time

What data warehouses provide:

Modern cloud data warehouses (BigQuery, Snowflake, Redshift) provide strong consistency for individual table operations — once a LOAD or INSERT completes, all subsequent queries see the updated data. However, in multi-step ETL pipelines, consistency across multiple tables is the pipeline's responsibility. If Pipeline A and Pipeline B both update related tables, consumers might see an inconsistent view if they query between the two updates.

Data engineering implication:

Design pipelines to update related tables atomically when possible. Use transaction-like patterns — write all related data in a single step, or use a completion flag that downstream consumers check before querying.

Q94. How do you handle a scenario where your pipeline must process the same source data in multiple different ways for different consumers?**Answer:****The challenge:**

Consumer A needs daily aggregated revenue. Consumer B needs raw transaction-level data. Consumer C needs a different aggregation by region. If you build a separate pipeline for each consumer, you extract and process the same source data three times — wasteful and inconsistent.

Solution — Shared foundation with consumer-specific layers:

- Layer 1 — Single raw extract: Extract source data once. Store in the raw/Bronze zone. All downstream processing starts from this single source of truth.
- Layer 2 — Single cleaned/enriched dataset: Apply common cleaning, deduplication, and enrichment once. Store in the Silver layer. This is the shared foundation.

- Layer 3 — Consumer-specific Gold tables: Build separate lightweight transformations from the shared Silver layer for each consumer's specific needs. Each consumer has their own Gold table, but they all derive from the same Silver source.

Benefits:

Source data is extracted once (lower cost, lower load on source). Cleaning logic is applied once (no inconsistency between consumers). Consumer-specific logic is isolated and independently maintainable.

Q95. What are the risks of using SELECT * in data pipeline queries and what should you do instead?

Answer:

Risks of SELECT *:

- Schema brittleness: When the source table adds a new column, SELECT * automatically includes it. If the destination table has a fixed schema, the pipeline breaks. If it does not, you silently load columns you never intended to load.
- Performance: In columnar storage systems, SELECT * reads all columns even when only a few are needed. This increases I/O, cost, and processing time proportionally to the number of unused columns.
- Hidden data exposure: SELECT * may inadvertently include PII or sensitive columns in a query result that is passed to a less secure destination.
- Breaking changes: If a column is removed from the source, SELECT * pipelines fail silently (the column just disappears from the output) — or noisily if downstream systems expected it.

Best practice:

Always explicitly name every column in pipeline queries: SELECT order_id, customer_id, revenue, created_at FROM orders. This makes the pipeline self-documenting, resilient to unexpected schema changes, and efficient.

Q96. How would you design a pipeline audit trail that proves data has not been tampered with?

Answer:

An audit trail pipeline design:

- Immutable raw zone: Raw data in the Bronze layer is write-once, never modified. Any transformation creates a new dataset in a new layer. The original raw data is the permanent proof of what the source sent.
- Cryptographic hashing: For each batch of raw data loaded, compute a hash (e.g., SHA-256) of the file or record set. Store the hash separately in an audit log table. Anyone can recompute the hash and compare — any tampering changes the hash.
- Pipeline execution log: Log every pipeline run with: timestamp, input data hash, transformation version (git commit), output data hash, row counts. This creates a complete verifiable chain of custody.
- Access logging: Log every read and write to sensitive data tables. Audit logs must themselves be write-only and access-controlled so they cannot be modified.
- Version control for transformations: All transformation logic in version control. Every pipeline run records the exact git commit hash of the code that ran. This proves the exact transformation applied.

Compliance benefit:

If audited, you can prove: this data came from this source at this time, was transformed by this code version, and produced this output — and none of it was modified after the fact.

Q97. What would you do if you discovered that your pipeline has been silently dropping 0.1% of records for the past 6 months?

Answer:

This is a serious incident. 0.1% sounds small but on 100 million records per day, that is 100,000 missing records daily — 18 million records missing over 6 months.

Immediate steps:

- Stop the bleeding: Identify and fix the bug causing the drops before running another pipeline run.
- Assess the impact: Which tables are affected? Which downstream reports, dashboards, and decisions relied on this data?
- Communicate proactively: Notify stakeholders that certain metrics have been undercounted. Do not hide the issue.

Recovery:

- Identify missing records: Compare source row counts vs destination row counts by date to quantify how many records are missing per day.
- Backfill from raw data: If you preserved raw data in the Bronze layer (you should have), re-process all 6 months from raw to recover the missing records.
- If raw data is gone: Request re-delivery from the source if possible. Document what cannot be recovered.

Prevention:

Add automated daily reconciliation checks (source row count vs destination row count) with alerts on any discrepancy > 0%. This incident should have been caught on day 1.

Q98. How do you design a pipeline that needs to handle both historical backfill and ongoing incremental loads without code duplication?

Answer:

A common mistake is building two separate pipelines — a 'historical load' script and an 'incremental load' script — with duplicated transformation logic that diverges over time.

Unified design approach:

- Single parameterized pipeline: Build one pipeline that accepts a date range parameter (start_date, end_date). For daily incremental runs: pass yesterday's date. For backfill: pass any historical date range.
- Same transformation code: The transformation logic is identical regardless of whether it is processing yesterday's data or data from 3 years ago. No separate backfill transformation.
- Idempotent partition overwrite: The same idempotent write strategy handles both cases — whether loading yesterday for the first time or reloading a historical partition for backfill.
- Configurable run modes: A single environment variable or config flag switches between modes: INCREMENTAL (auto-computes the next window) vs BACKFILL (accepts explicit date range from orchestration).

Benefit:

Every historical backfill is a stress test of the production incremental pipeline. Any bug in the transformation logic is caught during backfill and fixed once — not discovered separately in a distinct backfill script later.

Q99. What is the 'dual write' pattern and when is it used during data platform migrations?

Answer:

Dual write is a migration strategy where, during a transition period, every data write goes to both the old system and the new system simultaneously. This allows teams to validate the new system produces correct results before switching consumers over.

How it works:

- Phase 1: New system is built in parallel. All writes still go only to old system.
- Phase 2 (dual write): Writes go to both old and new systems. An automated reconciliation job compares outputs between them daily.
- Phase 3: After N weeks of reconciliation showing identical results, consumers are migrated to the new system one by one.
- Phase 4: Old system receives no more writes. It is kept in read-only mode as a fallback for some additional weeks.
- Phase 5: Old system decommissioned.

Why it is important:

It is the only safe way to migrate a production data system with zero data loss and zero surprise for consumers. Never do a hard cutover on day 1 of a new system.

Trade-off:

Dual write doubles the write load and storage cost during the migration period. This is an acceptable temporary cost for migration safety.

Q100. How do you ensure your data pipeline is production-ready before deploying it?

Answer — Production readiness checklist:

- Correctness: Has the transformation logic been verified against known expected outputs? Are there automated tests? Has the output been reconciled against the source for a sample date range?
- Idempotency: Has the pipeline been verified to produce identical results when run twice for the same date? Is it safe to retry?
- Performance: Has it been tested with production-scale data (not just a small dev sample)? Does it complete within the SLA window?
- Error handling: Does it handle bad records gracefully (DLQ) instead of crashing? Does it retry on transient failures?
- Monitoring and alerting: Are there alerts for failure, SLA breach, and data quality failures? Does someone get paged if it fails at 3 AM?
- Security: Are credentials managed securely (secrets manager, not hardcoded)? Does it use least-privilege access?
- Documentation: Is the pipeline documented — what it does, what it consumes, what it produces, who owns it, and what to do when it fails?
- Rollback plan: If the pipeline causes problems after deployment, what is the plan to rollback? Can you quickly revert to the previous version?

Key principle:

A pipeline is not done when it runs successfully once. It is production-ready only when it can run reliably day after day, fail gracefully, recover automatically, and be debugged efficiently by any team member — including someone who did not write it.

Interview Preparation Summary

Section	Focus Area	Questions
Section 1	Direct Conceptual Questions	Q1 – Q45 (45 questions)
Section 2	Scenario-Based Questions	Q46 – Q65 (20 questions)
Section 3	Best Practices & Architecture	Q66 – Q75 (10 questions)
Section 4	Performance Optimization	Q76 – Q85 (10 questions)
Section 5	Advanced & Tricky Questions	Q86 – Q100 (15 questions)
TOTAL		100 Questions

All questions are tool-agnostic and applicable to any data engineering role.

Good luck with your Data Engineering interview!